סדנה ליישום פרויקט הנדסת תוכנה (83538-01)

בניית סוכן נסיעות מבוסס בינה מלאכותית (Agentic Travel Planner)

מפגש 3: בניית סוכן אוטונומי עם LangGraph

א. הגדרת אבני הבניין

במפגש זה נעבור מ"תזמור ידני" למערכת אוטונומית שמנהלת את עצמה. במקום שאנחנו נכתוב לולאות `if/else`, נגדיר **מכונת מצבים (State Machine)** המורכבת משלושה אלמנטים:

1. **State:** ה"זיכרון" המשותף של הסוכן.
2. **Nodes:** הפונקציות שמבצעות את העבודה.
3. **Edges:** החוקים שקובעים את נתיב הזרימה.

1. הגדרת המצב (`state.py`)

ה-State הוא הזיכרון של הסוכן. כל צומת בגרף יכול לקרוא מהלוח הזה ולכתוב אליו.

```
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages

:(class AgentState(TypedDict
"""
.The central data structure shared by all nodes
messages': A history of the conversation. Annotated with'
''add_messages
to ensure new messages are appended instead of
.overwritten
.current_city': Tracks the user's destination across the session'
.total_budget': Tracks the user's financial limit'
"""
[messages: Annotated[list, add_messages
current_city: str
total_budget: float
```

2. צמתים (Nodes)

צומת הוא פונקציה שמקבלת את ה-State הנוכחי ומחזירה עדכון. הצמתים הם ה"תחנות" בגרף. בגרף הבסיסי שלנו יש שתי תחנות: המודל (Agent) והכלים (Tools).

לדוגמא:

```
# 3. Define the Agent Logic (Agent Node)

def call_model(state: AgentState):

    """

    Examines the current stat and decides whether to trigger a tool
call

    or provide a final answer to the user.

    """

    response = model.invoke(state["messages"])

    return {"messages": [response]}
```

3. קשתות (Edges)

קשת היא פונקציה הקובעת האם יש להפעיל כלי או האם ההרצה הגיעה לסיומה.

שני הסוגים העיקריים:

- קשת קבועה (Edge): מייצגת את הקשר הפשוט ביותר ומחברת בין צומת א' לצומת ב'.
מוגדרת בעזרת הקריאה:

```
builder.add_edge("node_a", "node_b")
```

ברגע ש-node_a מסיים, המערכת עוברת אוטומטית ל-node_b.

- קשת מותנית (Conditional Edge): משתמשת בפונקציית ניתוב (router) כדי להחליט על היעד.
המשמעות היא שהיעד נקבע בזמן אמן על סמך הstate.

```
builder.add_conditional_edges("node_a", routing_function_name)
```

סוג נוסף: parallel edges - מאפשרות להגדיר צומת אחד אשר ישלח מידע לכמה צמתים במקביל.
במקרה זה, פונקציית הניתוב של הקשת תחזיר רשימה של יעדים (למשל [" node_b" , " node_c"]) והגרף יריץ את שניהם במקביל.

```
# 4. Define Conditional Logic

def should_continue(state: AgentState):

    """
```

```
    Determines the next path in the graph based on the model's output.

    Returns 'tools' if the model wants to call a function, otherwise
'END'.

    """

    last_message = state["messages"][-1]

    if last_message.tool_calls:

        return "tools"

    return END
```

ב. בניית הגרף (The Graph Construction)

כעת נחבר את הכל יחד למבנה אחד.

```
import os

from typing import Annotated, TypedDict

from dotenv import load_dotenv

from langchain_google_genai import ChatGoogleGenerativeAI

from langgraph.graph import StateGraph, START, END

from langgraph.graph.message import add_messages

from langgraph.prebuilt import ToolNode

from tools import fetch_flights, fetch_hotels, calculate_trip_cost


load_dotenv()


# 1. Define the Shared State

class AgentState(TypedDict):

    """
```

```
    The central data structure shared by all nodes.

    'messages': Appends new messages to the history using the
add_messages reducer.

    'current_city': Tracks the target destination throughout the flow.

    'total_budget': Tracks the remaining funds for the trip.

    """

    messages: Annotated[list, add_messages]

    current_city: str

    total_budget: float


# 2. Initialize Model and Tools

tools = [fetch_flights, fetch_hotels, calculate_trip_cost]

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash",
temperature=0).bind_tools(tools)


# 3. Define the Agent Logic (Agent Node)

def call_model(state: AgentState):

    """

    Examines the current state and decides whether to trigger a tool
call

    or provide a final answer to the user.

    """

    response = model.invoke(state["messages"])

    return {"messages": [response]}


# 4. Define Conditional Logic
```

```python
def should_continue(state: AgentState):
    """
    Determines the next path in the graph based on the model's output.
    Returns 'tools' if the model wants to call a function, otherwise
'END'.
    """
    last_message = state["messages"][-1]
    if last_message.tool_calls:
        return "tools"
    return END


# 5. Build the Graph Workflow
builder = StateGraph(AgentState)


# Add Nodes
builder.add_node("agent", call_model)
builder.add_node("tools", ToolNode(tools))


# Define Connections
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", should_continue)
builder.add_edge("tools", "agent") # Loop back to the brain after tool
execution


# Compile the Graph
graph = builder.compile()
```

דגשים:

- שימו לב להוספת הצמתים והקשתות ולפונקציה המיוחדת: ToolNode - צומת מובנה מראש המריץ כלים בתהליכי עבודה (Workflows) של LangGraph https://docs.langchain.com/oss/python/langchain/tools#toolnode
- שימו לב שהגרף הוא מעגלי (`tools` חוזר ל-`agent`). זה מה שמאפשר לסוכן "לחשוב" שוב אחרי שהוא קיבל נתונים מה-DB.
- ה-`call_model` רק מקבל החלטות. ה-`tool_node` רק מריץ קוד. הם לא יודעים זה על קיומו של זה, הם רק מתקשרים דרך ה-`State`.
- שימו לב לSTART המיוחד: מגדיר איפה הגרף מתחיל
- שימו לב ל-END המיוחד: זהו סימן מוסכם ל-LangGraph לעצור את הריצה ולהחזיר את התשובה הסופית למשתמש.
- שימו לב לפעולת ההידור (קומפילציה) - פעולה זו מבצעת מספר בדיקות בסיסיות על מבנה הגרף (לדוגמה: זיהוי צמתים יתומים/מבודדים). אם היינו מוסיפים יישומיות של שמירת מצב (persistence) לאפליקציה שלנו באמצעות checkpointer (רכיב האחראי על שמירת ה-State (הזיכרון) במסד נתונים או בזיכרון המערכת), הוא היה מועבר פנימה בנקודה זו.

כעת נהפוך נהפוך את הסקריפט אינטראקטיבי:

הוסיפו את הקוד הבא לאחר שורת ההידור:

```
# --- INTERACTIVE EXECUTION ---



def run_agent():

    # thread_id identifies a specific conversation session

    config = {"configurable": {"thread_id": "student_session_01"}}



    print("--- Autonomous Travel Agent Started ---")



    while True:

        user_input = input("\nUser: ")

        if user_input.lower() in ["exit", "quit"]: break



        # Run the graph and stream the state updates

        for event in graph.stream({"messages": [("user", user_input)]},
config, stream_mode="values"):
```

```
            last_msg = event["messages"][-1]



            # Print the type of message and a snippet of its content

            msg_type = last_msg.__class__.__name__

            print(f"[{msg_type}] Content:
{str(last_msg.content)[:70]}...")



            # Print explicit state variables for tracking

            print(f"City: {event.get('current_city')} | Budget:
{event.get('total_budget')}")

            print("-" * 20)



if __name__ == "__main__":

    run_agent()
```

שימו לב: זהו סקריפט דוגמא שמדפיס את משתני המצב ותגובה קטועה לטובת הבנה. בהמשך, עליכם לשנות אותו כך שיתאים לחווית המשתמש שתרצו ליצור.

ב. מטלת השבוע: פיתוח לוגיקת הסוכן ומעבר לתשתית SQL

השבוע תרחיבו את הסוכן הבסיסי שבנינו למערכת רב-שכבתית המסוגלת להתמודד עם תרחישים מורכבים, למנוע תקלות נפוצות (כמו לופים אינסופיים) ולתמוך בלוגיקה עסקית מוגדרת מראש. במטלה זו נשדרג את ה"ארכיטקטורה" של הסוכן שלנו. המטרה היא להפוך אותו ממערכת פשוטה של "שאלה-תשובה" למערכת בעלת לוגיקה עסקית מובנית, תוך שדרוג הכלים לעבודה מול בסיס נתונים רלציוני.

שדרוג כלים ותשתית (עבודה עם SQLite)

כדי להתנסות בעבודה מול מקור נתונים תקני, עליכם לעדכן את הקובץ `tools.py` כך שישלוף מידע מתוך בסיס נתונים.

הריצו את סקריפט ה-`init_db.py` כדי ליצור את הקובץ `travel_agency.db` (הרצה חד פעמית בתיקיית הפרוייקט)

```
# init_db.py
import sqlite3
```

```python
def create_travel_db():
    conn = sqlite3.connect("travel_agency.db")
    cursor = conn.cursor()

    # 1. Create Tables
    cursor.execute("DROP TABLE IF EXISTS hotels")
    cursor.execute("DROP TABLE IF EXISTS flights")
    cursor.execute("DROP TABLE IF EXISTS activities")

    cursor.execute("""
        CREATE TABLE hotels (
            id INTEGER PRIMARY KEY,
            city TEXT,
            name TEXT,
            price_per_night INTEGER,
            stars INTEGER
        )
    """)

    cursor.execute("""
        CREATE TABLE flights (
            id INTEGER PRIMARY KEY,
            origin TEXT,
            destination TEXT,
            airline TEXT,
            price INTEGER,
            flight_number TEXT
        )
    """)

    cursor.execute("""
        CREATE TABLE activities (
            id INTEGER PRIMARY KEY,
            city TEXT,
            name TEXT,
            category TEXT,
            price INTEGER
        )
    """)

    # 2. Insert Expanded Sample Data

    # --- Hotels ---
```

```
    hotels = [
        ('paris', 'Hotel de Ville', 150, 3),
        ('paris', 'Luxury Ritz', 600, 5),
        ('paris', 'Ibis Budget Paris', 85, 2),
        ('london', 'The Savoy', 450, 5),
        ('london', 'Premier Inn London', 120, 3),
        ('tokyo', 'Shibuya Capsule', 50, 2),
        ('tokyo', 'Park Hyatt Tokyo', 700, 5),
        ('new york', 'The Plaza', 850, 5),
        ('new york', 'Broadway Hotel', 190, 3),
        ('berlin', 'Berlin Central Hostel', 40, 1),
        ('berlin', 'Hilton Berlin', 220, 4)
    ]
    cursor.executemany("INSERT INTO hotels (city, name,
price_per_night, stars) VALUES (?, ?, ?, ?)", hotels)

    # --- Flights ---
    flights = [
        ('TLV', 'Paris', 'El Al', 350, 'LY321'),
        ('TLV', 'Paris', 'Air France', 420, 'AF123'),
        ('TLV', 'London', 'British Airways', 450, 'BA164'),
        ('TLV', 'London', 'Virgin Atlantic', 390, 'VS100'),
        ('TLV', 'Tokyo', 'EL AL', 950, 'LY091'),
        ('TLV', 'Tokyo', 'Emirates', 820, 'EK312'),
        ('TLV', 'New York', 'United', 750, 'UA445'),
        ('TLV', 'Berlin', 'Lufthansa', 280, 'LH909'),
        ('TLV', 'Berlin', 'Ryanair', 110, 'FR101')
    ]
    cursor.executemany("INSERT INTO flights (origin, destination,
airline, price, flight_number) VALUES (?, ?, ?, ?, ?)", flights)

    # --- Activities ---
    activities = [
        ('paris', 'Louvre Museum', 'Culture', 20),
        ('paris', 'Eiffel Tower', 'Sightseeing', 35),
        ('paris', 'Disneyland', 'Family', 95),
        ('london', 'London Eye', 'Sightseeing', 30),
        ('london', 'British Museum', 'Culture', 0),
        ('tokyo', 'Robot Cafe', 'Entertainment', 60),
        ('tokyo', 'Mount Fuji Day Trip', 'Nature', 120),
        ('new york', 'Statue of Liberty', 'Sightseeing', 25),
        ('berlin', 'Berlin Wall Tour', 'History', 15),
        ('berlin', 'Techno Club Entry', 'Nightlife', 25)
```

```
    ]
    cursor.executemany("INSERT INTO activities (city, name, category,
price) VALUES (?, ?, ?, ?)", activities)

    conn.commit()
    conn.close()
    print("Database 'travel_agency.db' created with expanded data!")

if __name__ == "__main__":
    create_travel_db()
```

כעת, עדכנו את הפונקציות ב-`tools.py` לשימוש ב-SQL. מצורפת דוגמא:

```
import sqlite3
import os
from langchain_core.tools import tool

DB_PATH = "travel_agency.db"

def _run_query(query: str, params: tuple = ()):
    """Helper function to execute a SQL query and return results."""
    if not os.path.exists(DB_PATH):
        return "Error: Database file not found. Please run init_db.py
first."

    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    try:
        cursor.execute(query, params)
        results = cursor.fetchall()
        # Get column names to return a list of dictionaries
        column_names = [description[0] for description in
cursor.description]
        return [dict(zip(column_names, row)) for row in results]
    except sqlite3.Error as e:
        return f"Database error: {e}"
    finally:
        conn.close()

@tool
def fetch_flights(origin: str, destination: str):
    """
    Search for available flights between two locations in the database.
```

```
    Input:
    - origin: 3-letter airport code (e.g., 'TLV', 'JFK', 'LHR').
    - destination: Full city name (e.g., 'Paris', 'London', 'Tokyo').
    Returns: List of matching flights with airline, price, and flight
number.
    """
    # Using LOWER() to ensure case-insensitive matching in the database
    query = "SELECT airline, price, flight_number FROM flights WHERE
LOWER(origin) = ? AND LOWER(destination) = ?"

    # Pre-processing inputs to match the database format
    origin_param = origin.strip().lower()
    dest_param = destination.strip().lower()

    print(f"DEBUG: Searching flights from '{origin_param}' to
'{dest_param}'") # For your tracking

    matches = _run_query(query, (origin_param, dest_param))

    if not matches or isinstance(matches, str):
        return f"No flights found from {origin} to {destination}. Make
sure to use airport codes for origin."
    return matches

@tool
def fetch_hotels(city: str, max_price: int = None):
    """
    Find hotels in a specific city from the database.
    Input: city name (string), max_price (optional integer).
    """
    query = "SELECT name, price_per_night, stars FROM hotels WHERE
LOWER(city) = ?"
    params = [city.strip().lower()]

    if max_price is not None:
        query += " AND price_per_night <= ?"
        params.append(max_price)

    matches = _run_query(query, tuple(params))

    if not matches or isinstance(matches, str):
        return f"No hotels found in {city} meeting those criteria."
    return matches
```

```
@tool
def calculate_trip_cost(flight_price: float, hotel_price_per_night:
float, duration_days: int):
    """
    Calculates the total cost for a trip including flight and hotel
stay.
    Input: flight_price (float), hotel_price_per_night (float),
duration_days (int).
    """
    try:
        total_hotel = float(hotel_price_per_night) * int(duration_days)
        total_grand = float(flight_price) + total_hotel

        return {
            "breakdown": {
                "flight": flight_price,
                "hotel_total": total_hotel,
                "days": duration_days
            },
            "total_estimate": total_grand,
            "currency": "USD"
        }
    except (ValueError, TypeError):
        return "Error: Please provide valid numbers for prices and
duration."
```

חלק 2: שכלול הגרף (Nodes & Edges)

עליכם להרחיב את ה-`StateGraph` שלכם ולהגדיר "אישיות" ולוגיקה לסוכן.

- הוספת צמתים (Nodes): הוסיפו לפחות שני צמתים חדשים המבצעים לוגיקה פנימית.
  - שימו לב, כאשר אתם מוסיפים צמתים חדשים לגרף, עליכם להגדיר מחדש את הקשתות (Edges) שמחברות אותם למערכת. צומת שלא מחובר בקשת כניסה (Inbound) וקשת יציאה (Outbound) לא יופעל לעולם או "יתקע" את הסוכן. דפוסי חיבור לדוגמא: חיבור טורי (לדוגמא, הכנסת צומת בדיקה בין צומת אחד לצומת שני), פיצול\ניתוב (שימוש בconditional edges), עיבוד מקדים (צומת שרץ תמיד לפני צומת "מרכזי").
  - חשבו על צמתים הנדרשים למימוש הלוגיקה שלכם
- חיבור קשתות (Edges):
  - השתמשו ב-Fixed Edges לזרימה קבועה.
  - השתמשו ב-Conditional Edges לקבלת החלטות בזמן אמת.
- מנגנוני הגנה: עליכם להטמיע פתרונות לבעיות אפשריות בתוך מבנה הגרף, לדוגמא: הוסיפו לוגיקה למניעת לופ אינסופי (למשל: הגבלת מספר הפעמים שהסוכן יכול לגשת לכלים בשיחה אחת - רמז

step count) או צרו קשת המתמודדת עם מצב שבו כלי מחזיר שגיאה (למשל עיר שלא קיימת ב-DB). במקום שהסוכן ינסה שוב ושוב, עליו לעצור ולהסביר למשתמש את הבעיה.

<u>למפגש הבא עליכם להכין:</u>

1. מצגת הכוללת:
   - הסבר קצר על ה"אישיות" והלוגיקה של הסוכן
   - תרשים של ארכיטקטורת הגרף שלכם (State, Nodes, Edges, Conditional Edges)
2. דמו: הכינו 3-5 שאילתות להרצה חיה (חשבו על שאילתות שידגימו את הלוגיקה שמימשתם ואת מקרי הקצה).

ד. הכנה למפגש 4

במימוש הנוכחי הסוכן שלכם כרגע "נולד מחדש" בכל פעם שאתם מריצים את הסקריפט. במפגש הבא ניתן לו זיכרון קבוע שישמר גם אחרי סגירת התוכנה.

לקראת המפגש הבא:

1. קראו על האובייקט **SqliteSaver**.
2. בדקו בתיעוד איך LangGraph משתמש בפרמטר: `thread_id` כדי להפריד בין משתמשים או שיחות שונות.
   חשבו על תרחיש שבו המשתמש עוצר את השיחה באמצע וחוזר אליה אחרי יום. אילו נתונים ב-AgentState חייבים להישמר בבסיס הנתונים כדי שהסוכן ידע להמשיך בדיוק מאותה נקודה?

Appendix: Q&A

1. **Why is my messages list growing so long? Can I clear it?**
   In LangGraph, the messages list is your primary memory. We use Annotated[list, add_messages] to ensure the LLM has the full context of the conversation. If you clear it, the agent will "forget" what the user just said. Note: If you want to keep the context short, you can implement a "Summarization Node" that replaces old messages with a single summary message.
2. **How do I update the current_city field automatically?**
   You can create a specialized node that runs at the beginning of the graph. This node inspects the latest user message and updates the state variables.

```
def extract_metadata(state: AgentState):

    """

    Scans the last message for city names or budget mentions.

    This helps the agent stay focused on the destination.

    """

    last_message = state["messages"][-1].content.lower()
```

```
    # Simple logic for demonstration (In production, use an LLM for
this)
    if "paris" in last_message:
        return {"current_city": "Paris"}
    if "london" in last_message:
        return {"current_city": "London"}


    return {} # No update needed
```

3. **What happens if a tool returns an error?**
   By default, the `ToolNode` will return the error string as the tool output. The `agent` node will then receive this error in its history. Note: You can add a *Conditional Edge* that checks if the tool output contains the word "Error". If it does, instead of going back to the `agent`, you can route to a `human_assistance` node.
4. **Why does the graph loop infinitely between `agent` and `tools`?**
   This usually happens because of the `should_continue` logic. If the model keeps generating `tool_calls` (perhaps because it's not satisfied with the answer or the tool keeps failing), it will loop forever. Note: Check your `system_prompt`. Ensure you tell the model: "If you have all the information, provide a final answer and do not call any more tools."
5. **Can I have multiple "agent" nodes?** Yes! This is a core feature of LangGraph. You can have a `travel_agent` node for bookings and a `safety_advisor` node for travel warnings. You simply use a *Conditional Edge* to route the user's question to the correct specialized node.

```
def router(state: AgentState):
    """
    Routes to different specialized agents based on user intent.
    """
    user_query = state["messages"][-1].content.lower()
    if "safe" in user_query or "visa" in user_query:
        return "safety_expert"
```

```
    return "travel_agent"
```